

UNIVERSITÀ POLITECNICA DELLE MARCHE

INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

Applicazione del Reinforcement Learning su un robot antropomorfo



Corso di
DYNAMICS AND CONTROL OF INTELLIGENT ROBOTS
AND VEHICLES

Anno accademico 2024-2025

Studenti:

Andrea Perugini

Enzo Cingoli

Gabriel Vladut Popa

Professore:

Andrea Bonci

Dottorando:

Alessandro Di Biase



Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione	3
1 Sistema	4
1.1 Il Manipolatore TM5-900	4
1.2 Modello Matematico e Simulazione	4
1.3 Interfaccia di Controllo	5
2 Hardware	7
2.1 Architettura di Simulazione	7
2.2 Componenti Simulati	7
2.2.1 Attuatori (Joint Controllers)	8
2.2.2 Sensori (Joint State Interface)	8
2.3 Schema dei collegamenti	8
3 Software	10
3.1 Installazione	10
3.1.1 Requisiti di Sistema	10
3.1.2 Procedura di Installazione	10
3.1.3 Configurazione dell'Ambiente	11
3.2 Pipeline di esecuzione	11
3.3 Diagramma di flusso	13
3.4 Gestione pacchetti ROS 2	14
3.5 Dettaglio dei pacchetti	16
3.5.1 my_environment_pkg	16
3.5.2 my_sphere_pkg (pacchetto per il goal)	20
3.5.3 my_obstacle_pkg (pacchetto per gli ostacoli)	21
3.5.4 tm5_900 (descrizione robot & controller)	22
3.6 Funzionamento complessivo	26
4 Test e risultati	27
4.1 Configurazione Sperimentale	27
4.1.1 Funzioni di Reward	27
4.2 Test 1: Curriculum Learning con SAC	28
4.2.1 Analisi Critica dei Risultati	28
4.3 Test 2: Success Threshold Annealing con SAC + HER	30
4.3.1 Analisi dei Risultati	30
4.3.2 Validazione Finale	31
4.4 Problematiche Riscontrate e Lezioni Apprese	32
4.4.1 Problematiche	32
4.4.2 Lezioni Apprese	32

Introduzione

Il presente progetto si focalizza sull'applicazione di tecniche di *Deep Reinforcement Learning* (DRL) per il controllo di un manipolatore robotico antropomorfo, nello specifico il modello **TM5-900**. Il task assegnato al gruppo consiste nel cosiddetto "Reach Task": il robot deve imparare autonomamente a muovere il proprio *end-effector* per raggiungere un target sferico generato casualmente all'interno del suo spazio di lavoro.

L'attività è stata svolta interamente in ambiente simulato, utilizzando il framework **ROS 2** (distribuzione Foxy) in combinazione con il simulatore fisico **Gazebo**. Questa scelta ha permesso di modellare la cinematica e la dinamica del braccio robotico in sicurezza prima di un'eventuale, futura, implementazione su hardware reale. L'algoritmo di apprendimento implementato è il **Soft Actor-Critic (SAC)**, scelto per la sua efficienza nell'esplorazione e stabilità in spazi di azione continui. A questo è stata affiancata la tecnica **Hindsight Experience Replay (HER)**, fondamentale per affrontare la problematica della sparsità del reward tipica dei compiti di raggiungimento precisi.

Il lavoro è stato svolto in autonomia dal gruppo, curando l'intera pipeline: dalla configurazione dell'ambiente di simulazione (inclusa la gestione degli ostacoli e dei target dinamici), allo sviluppo dei nodi ROS 2 per il training, fino all'analisi dei dati di log. Non vi sono state interazioni dirette con moduli sviluppati da altri gruppi, pertanto l'architettura presentata è autoconsistente.

Di seguito viene riportata una breve descrizione dell'organizzazione della relazione:

- Nel **Capitolo 1** viene descritto il sistema robotico TM5-900 oggetto di studio, il suo modello cinematico e le equazioni fondamentali che ne governano il moto.
- Il **Capitolo 2** illustra la configurazione dell'ambiente simulato, descrivendo i sensori virtuali utilizzati (es. joint states) e l'interfacciamento tra il controllore e il modello fisico in Gazebo.
- Nel **Capitolo 3** viene dettagliata l'architettura software, presentando i pacchetti ROS 2 sviluppati (environment, sphere management, agent training), i diagrammi di flusso dell'algoritmo e le configurazioni specifiche adottate per il SAC+HER.
- Il **Capitolo 4** è dedicato all'analisi sperimentale: vengono discussi i risultati del training, le metriche di successo ottenute e le criticità emerse, con particolare attenzione all'impatto della simulazione della gravità sulle performance del controllore.
- Infine, nelle **Conclusioni**, si riassumono i traguardi raggiunti e si propongono sviluppi futuri per migliorare la robustezza del sistema.

1 Sistema

In questo capitolo viene descritto il sistema robotico utilizzato per l'attività di progetto, focalizzandosi sulla sua rappresentazione in ambiente simulato e sulle interfacce di controllo predisposte per l'algoritmo di apprendimento.

1.1 Il Manipolatore TM5-900

Il robot oggetto di studio è il **TM5-900** della Figura 1.1, un manipolatore antropomorfo collaborativo (cobot) a 6 gradi di libertà (DoF), ampiamente utilizzato in ambito industriale per compiti di assemblaggio, pick-and-place e asservimento macchine.

Per lo svolgimento del task, non disponendo dell'hardware fisico, si è fatto ricorso a una replica digitale fedele fornita tramite pacchetti ROS 2 ufficiali o adattati dalla comunità open source e dal materiale didattico. L'integrazione in ambiente simulato ha permesso di testare l'algoritmo di Reinforcement Learning in condizioni di sicurezza, evitando i rischi di danneggiamento associati alle fasi iniziali dell'esplorazione (trial-and-error).



Figure 1.1: *Manipolatore robotico TM5900*

1.2 Modello Matematico e Simulazione

Piuttosto che derivare manualmente le equazioni del moto (Lagrangiana o Newton-Eulero), l'approccio adottato sfrutta le capacità del motore fisico di **Gazebo** di risolvere la dinamica del sistema in tempo reale. Il modello matematico del robot è definito implicitamente attraverso lo standard **URDF** (Unified Robot Description Format), generato a partire da macro **XACRO**.

I file di descrizione (.xacro) utilizzati nel progetto contengono tutte le informazioni necessarie per la simulazione fisica e cinematica:

- **Catena Cinematica:** Definizione dei link (bracci) e dei joint (giunti), con relative trasformazioni coordinate (origini e orientamenti).
- **Proprietà Dinamiche:** Per ogni link sono specificati massa e tensore d'inerzia, dati fondamentali per una corretta simulazione delle forze e della gravità.
- **Limiti Fisici:** Sono imposti limiti precisi su posizione (angolo massimo/minimo), velocità e sforzo (coppia massima) per ogni giunto, rispecchiando le specifiche tecniche del robot reale.
- **Geometrie:** Il modello include mesh dettagliate per la visualizzazione (visual) e mesh per il calcolo efficiente delle collisioni (collision).

L'utilizzo di questi file standardizzati garantisce che il simulatore risolva la cinematica diretta e inversa e la dinamica dei corpi rigidi in modo coerente, permettendo al gruppo di lavoro di concentrarsi sulla logica di controllo di alto livello.

Nella Figura 1.2 è presente una vista di Gazebo, contenente il braccio simulato e la sfera generata nello spazio.

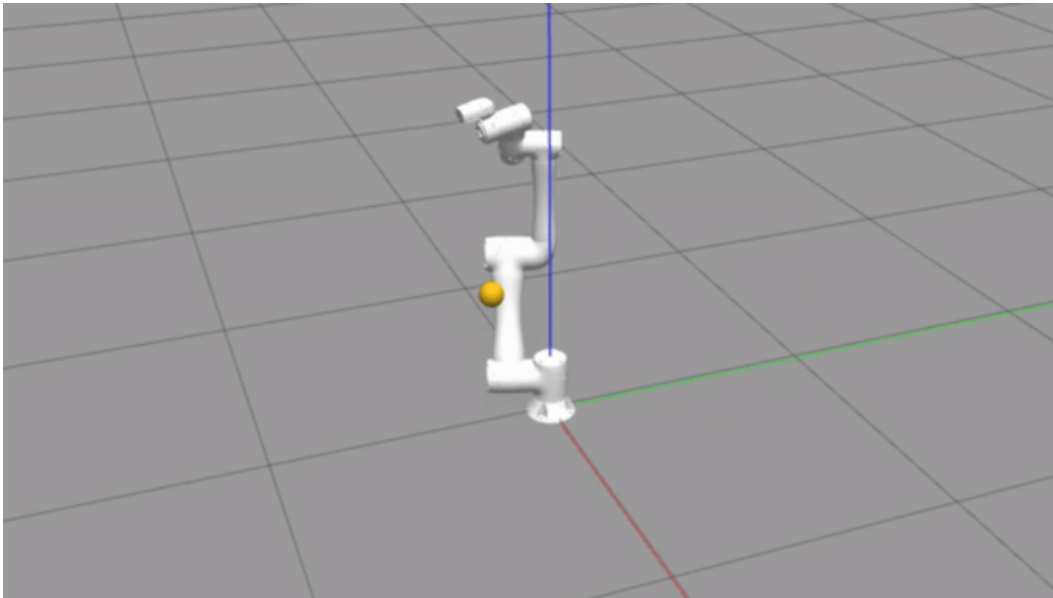


Figure 1.2: *Vista di Gazebo*

1.3 Interfaccia di Controllo

Il controllo del sistema avviene attraverso l'infrastruttura di comunicazione di ROS 2. Il sistema è stato configurato per esporre le interfacce necessarie all'agente SAC per:

1. **Osservazione dello Stato:** Lettura in tempo reale delle posizioni e delle velocità angolari dei 6 giunti tramite i topic di *Joint States*.

2. **Invio Azioni:** Trasmissione dei comandi di controllo calcolati dalla rete neurale. A seconda della configurazione specifica del training, l'agente agisce inviando riferimenti di posizione ai controllori dei giunti simulati in Gazebo.

Questa astrazione software permette di disaccoppiare l'algoritmo di apprendimento dalla complessità meccanica del robot: per l'agente, il "sistema" è un ambiente che riceve un vettore di azioni e restituisce un vettore di stato e un reward, indipendentemente dal fatto che dietro ci sia un motore fisico simulato o dei motori reali.

2 Hardware

In questo progetto non è stato utilizzato hardware robotico fisico. L'intera attività si è svolta in ambiente simulato. Di seguito vengono descritte le risorse di calcolo utilizzate e la configurazione dei componenti virtuali del robot.

2.1 Architettura di Simulazione

Il sistema è stato eseguito su una workstation configurata per supportare il carico computazionale della simulazione fisica e dell'addestramento della rete neurale.

Le specifiche dell'ambiente di esecuzione sono le seguenti:

- **Sistema Operativo:** Ubuntu 20.04 LTS.
- **Middleware:** ROS 2 Foxy Fitzroy.
- **Simulatore:** Gazebo (versione 11).

L'hardware del robot (motori, sensori, cablaggi) è interamente emulato dal motore fisico di Gazebo, che si occupa di calcolare le interazioni dinamiche e di fornire i feedback al controllore.

2.2 Componenti Simulati

Il modello del manipolatore TM5-900 in Gazebo è composto dai seguenti elementi funzionali che sostituiscono le controparti fisiche:

- **Base:** rappresentata dal link `base_link`, include la massa della base e un cilindro di inerzia lungo l'asse y . Serve da supporto per il braccio e ospita il giunto di collegamento fisso con il sistema ROS2.
- **Spalla (Shoulder):** composta dai link `shoulder_1_link` e il giunto `shoulder_1_joint`, con un secondo giunto `shoulder_2_joint` che collega il primo segmento del braccio. Entrambi i giunti sono revolute, con limiti di rotazione definiti e controller di sicurezza opzionali.
- **Braccio (Arm):** segmenti `arm_1_link` e `arm_2_link` collegati rispettivamente dai giunti `elbow_joint`. Ogni segmento ha lunghezza e massa specificate, e un cilindro di inerzia lungo l'asse y per rappresentare la dinamica del braccio.
- **Polso (Wrist):** composto dai link `wrist_1_link`, `wrist_2_link` e `wrist_3_link`, collegati da giunti revolute (`wrist_1_joint`, `wrist_2_joint`, `wrist_3_joint`) con limiti di rotazione e damping impostati. I segmenti del polso includono i rispettivi cilindri di inerzia sugli assi y o z a seconda della geometria.

- **Flangia e tool center point (TCP):** il link `flange_link` è collegato con giunto fisso `tool0` ai giunti del polso
- **Trasmissioni e gazebo plugins:** il modello include macro XACRO per la trasmissione dei giunti, configurazioni interne del robot e sensori (ad esempio, bumper sensor), che permettono la simulazione dinamica e il controllo tramite ROS2.
- **Parametri geometrici e dinamici:** lunghezze dei segmenti, masse e raggi dei cilindri, offset dei giunti e limiti di sicurezza sono definiti come proprietà XACRO, per facilitare modifiche parametriche e garantire coerenza tra modello fisico e simulazione.

Questo modello permette di simulare in Gazebo il comportamento del TM5-900, comprese le dinamiche dei giunti, l'inerzia dei segmenti e l'interazione con l'ambiente, fornendo un ambiente realistico per il controllo tramite ROS2 e l'applicazione di algoritmi di Reinforcement Learning.

2.2.1 Attuatori (Joint Controllers)

I motori dei 6 giunti sono simulati tramite plugin di controllo di ROS. Questi ricevono in ingresso i comandi di posizione inviati dall'agente e applicano le forze necessarie ai link del modello URDF per generare il movimento desiderato.

2.2.2 Sensori (Joint State Interface)

Al posto degli encoder fisici, il simulatore espone un'interfaccia di stato (`joint_states`) che pubblica in tempo reale:

- Posizione angolare dei giunti (radianti).
- Velocità angolare dei giunti (radianti/secondo).

Questi dati costituiscono l'input principale per l'osservazione dello stato da parte dell'agente.

2.3 Schema dei collegamenti

Non essendoci collegamenti elettrici fisici, lo "schema dei collegamenti" è rappresentato dal grafo di comunicazione di ROS 2. I nodi software (il nodo di training e il nodo ambiente) si collegano al "robot" (il server di simulazione Gazebo) tramite topic e servizi standard:

- **Topic comandi e actions:** Trasporta le azioni calcolate dalla rete neurale ai controller simulati.
- **Topic stati:** Trasporta le letture dei sensori virtuali dal simulatore all'agente.
- **Servizi di reset:** Permettono di riportare il robot e il target in una configurazione iniziale valida all'inizio di ogni episodio.

La Figura 2.1 mostra una rappresentazione concettuale di questo flusso di dati, che sostituisce il cablaggio fisico tradizionale.

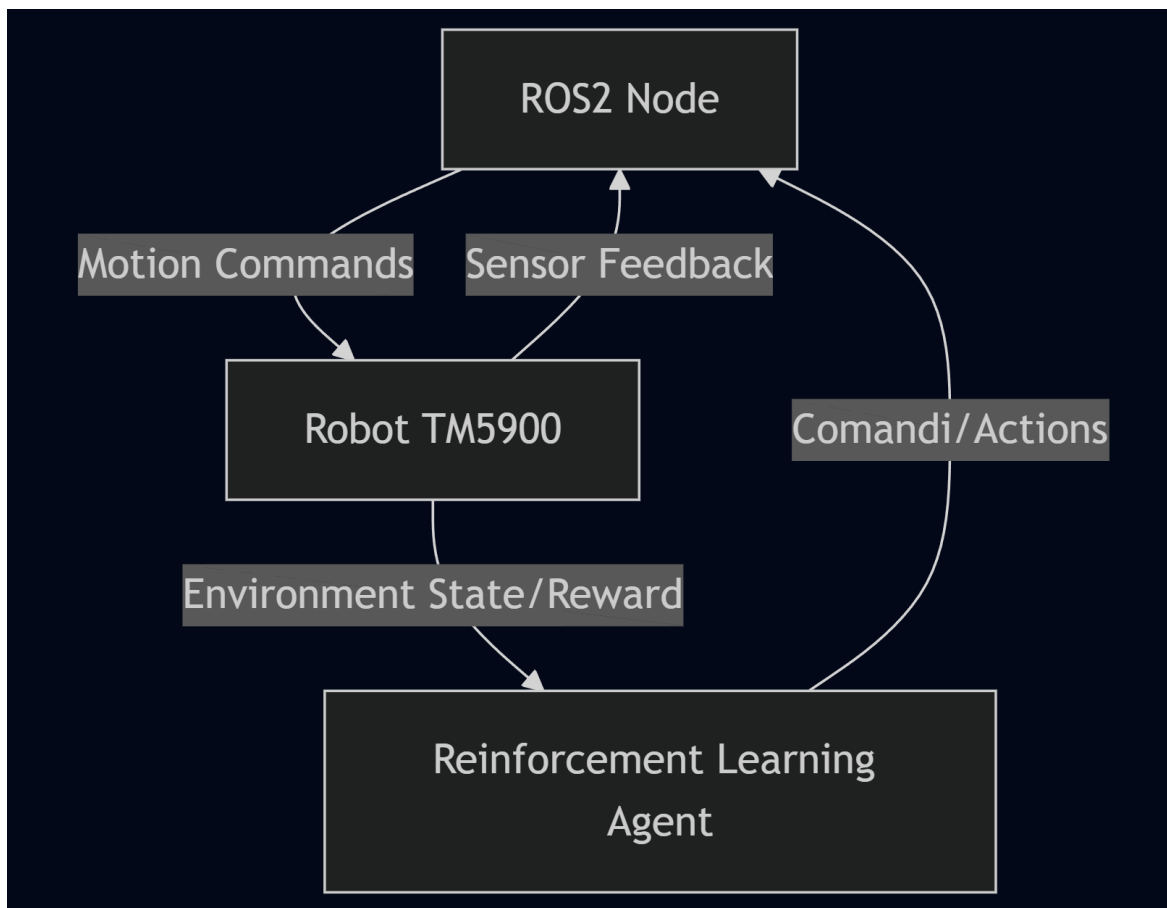


Figure 2.1: Schema logico delle connessioni tra Agente RL, Simulatore Gazebo e Ros2

3 Software

In questo capitolo viene descritta l'architettura software sviluppata per il progetto. Il codice è stato scritto principalmente in **Python 3.8**, sfruttando le librerie di **ROS 2 Foxy** per la comunicazione e **PyTorch** per l'implementazione delle reti neurali dell'agente.

3.1 Installazione

3.1.1 Requisiti di Sistema

Per eseguire il progetto sono necessari i seguenti requisiti:

- Sistema operativo: **Ubuntu 20.04**
- **ROS2 Foxy**
- Librerie: le **versioni esatte** delle librerie utilizzate saranno installate in seguito, tramite il file requirements.txt interno al github del progetto.

3.1.2 Procedura di Installazione

Un tutorial sull'installazione di ROS2 è riportato al seguente link: <https://docs.ros.org/en/foxy/Installation/Ubuntu-Install-Debians.html>

1. Creazione del Workspace ROS2

Aprire un terminale ed eseguire i seguenti comandi:

```
source /opt/ros/foxy/setup.bash
mkdir -p ros2_ws/src
cd ros2_ws
colcon build
```

2. Installazione delle Dipendenze ROS2

Installare i pacchetti ROS2 necessari:

```
sudo apt update
sudo apt install -y \
    python3-vcstool \
    ros-foxy-test-msgs \
    ros-foxy-control-toolbox \
    ros-foxy-gazebo-ros-pkgs \
    ros-foxy-xacro \
    ros-foxy-joint-state-publisher-gui \
    ros-foxy-ackermann-msgs
```

3. Installazione di Python e delle Librerie Necessarie

Installare Python 3 e i pacchetti richiesti:

```
sudo apt install -y python3-pip
pip3 install gymnasium torch
```

4. Installazione di ros2_control e gazebo_ros2_control

Scaricare e compilare i pacchetti di controllo:

```
cd ~/ros2_ws
wget https://raw.githubusercontent.com/ros-controls/ros2_control/refs
/heads/foxy/ros2_control/ros2_control.repos
vcs import src < ros2_control.repos
cd src
git clone -b foxy https://github.com/ros-simulation/
gazebo_ros2_control.git
cd ..
colcon build --symlink-install
```

5. Installazione del Pacchetto del Progetto

Clonare il repository del progetto e compilare:

```
cd ~/ros2_ws/src
git clone https://github.com/Gabri2023/
Dynamics_ROS2_ReachTask_tm5_900.git
cd ..
pip install -r requirements.txt
colcon build --symlink-install
```

3.1.3 Configurazione dell'Ambiente

Dopo l'installazione e compilazione con il comando colcon, è necessario eseguire il source del workspace in ogni nuovo terminale:

```
source ~/ros2_ws/install/setup.bash
```

Suggerimento: Per evitare di dover eseguire questo comando manualmente ogni volta, è possibile aggiungerlo al file ~/. bashrc:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/. bashrc
```

3.2 Pipeline di esecuzione

La pipeline operativa del progetto, così come dichiarata nel README del repository, è strutturata secondo i seguenti passi, da eseguire in successione:

1. Build e setup del workspace (ripetere in due terminali separati A e B):

```
source /opt/ros/foxy/setup.bash && colcon build && source install
/setup.bash
```

2. Lancio dell'ambiente simulato (nel terminale A):

```
ros2 launch my_environment_pkg my_environment.launch.py
```

3. Avvio della fase di addestramento (nel terminale B):

```
ros2 run my_environment_pkg run_environment
```

NOTA: durante la fase di addestramento al punto 3 avviene anche il salvataggio dei log di training nella cartella `checkpoints/train_8`, all'interno file `Monitoraggio.csv` (queste cartelle si creano in automatico con il comando al punto 3). Questo è importante per poter osservare, durante o al termine dell'allenamento, l'andamento del reward e dei successi.

Per graficare l'andamento dei log di addestramento sarà necessario eseguire il seguente script (si deve aver prima avviato almeno un training):

```
my_environment_pkg/plot_train.py
```

4. Dopo aver ottenuto un modello dal training (train completato o in pausa), è possibile fare il test dell'agente addestrato con il comando (nel terminale B):

```
ros2 run my_environment_pkg test_agent
```

NOTA: al termine del test al punto 4, si creerà automaticamente un file alla locazione `checkpoints/train_8/Test_numeroTest.csv`. Esso contiene numerosi dati utili all'analisi di performance sul modello addestrato, come riportato nella Tabella 3.1. Il codice con il quale tali log sono stati generati è contenuto nel file `my_environment_pkg/test_agent.py`

I passi devono essere eseguiti in successione, non simultaneamente. In particolare, per poter eseguire un test al punto 4, deve essere completato almeno un train del modello (avere dei pesi salvati nella cartella `checkpoints`). Questa organizzazione consente di separare chiaramente le fasi di simulazione, training e valutazione, migliorando la leggibilità e la riproducibilità degli esperimenti.

Table 3.1: *Descrizione delle variabili registrate durante il test*

Colonna	Descrizione
Epoche_modello	Numero di episodi sui quali il modello è stato allenato (valore fisso)
Num_episodio_attuale	Numero dell'episodio di test in esecuzione
Num_episodi_da_fare	Numero totale di episodi da completare durante la fase di test (valore fisso)
Dist_EE_target_iniziale	Distanza iniziale tra l'end-effector e il target
Dist_EE_target_attuale	Distanza corrente tra l'end-effector e il target
X_target, Y_target, Z_target	Coordinate cartesiane del target nello spazio
Numero_step	Step corrente dell'Num_episodio_attuale di test in esecuzione
Ricompensa_episodio_attuale	Ricompensa cumulativa ottenuta fino al Num_episodio_attuale
Conteggio_successi_globale	Numero totale di successi accumulati fino al Num_episodio_attuale
Tasso_successo_globale	Rapporto tra Conteggio_successi_globale e Num_episodio_attuale
Angolo_primo_giunto ... Angolo_sesto_giunto	Valori angolari dei sei giunti del manipolatore
Stato_home_position	Variabile booleana: True se il robot si trova in posizione home
Target_raggiunto	Variabile booleana: True se il target è stato raggiunto nello step corrente

Per chiarimento, inseriamo di seguito una breve legenda per decifrare al meglio la 3.1:

- **Step:** singolo movimento del robot, dalla posizione X alla posizione X'.
- **Episodio:** ciclo completo di lavoro contenente il reset dell'environment, esecuzione di una sequenza di step, calcolo reward.
- **Epoche:** nel nostro caso coincide con il concetto di episodio. Lo usiamo per indicare il numero di episodi di train del modello, la memoria storica.

3.3 Diagramma di flusso

Il funzionamento generale del sistema si basa sul ciclo di interazione tipico dell'apprendimento per rinforzo, sincronizzato con i tempi della simulazione. La Figura 3.1 illustra il loop principale di addestramento:

1. Ambiente (Robot TM5900) invia l'osservazione corrente s_t all'agente.
2. SAC+HER Agent sceglie un'azione a_t basata sulla policy network.
3. L'azione viene eseguita sul robot, ottenendo il nuovo stato s_{t+1} e reward.
4. HER (Hindsight Experience Replay) genera esperienze aggiuntive basate sugli obiettivi raggiunti e li inserisce nel replay buffer.
5. Il replay buffer campiona batch di traiettorie per aggiornare la policy network.
6. Il ciclo si ripete fino al completamento dell'episodio o al raggiungimento dell'obiettivo con un'elevata confidenza.

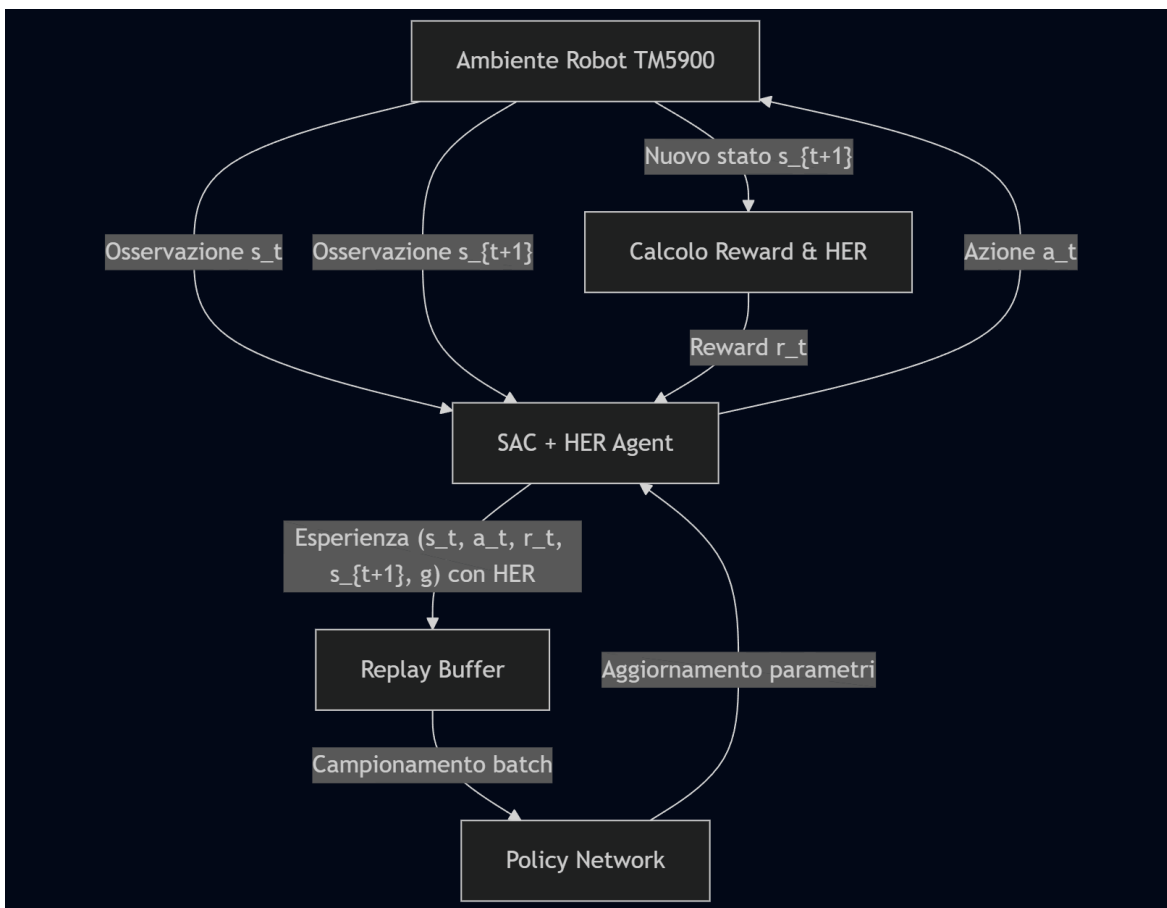
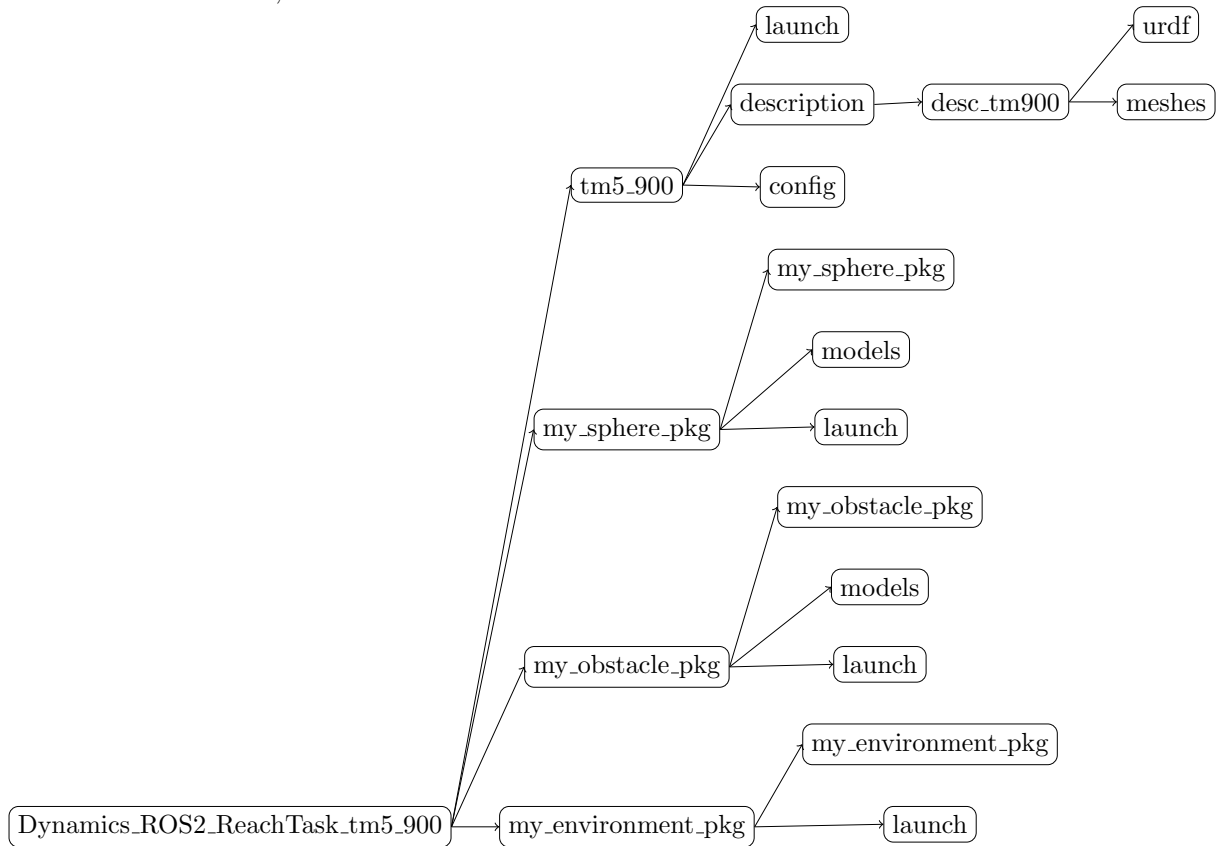


Figure 3.1: *Diagramma di flusso del ciclo di addestramento SAC+HER*

3.4 Gestione pacchetti ROS 2

Il workspace è stato organizzato in pacchetti modulari per separare la logica di simulazione da quella di apprendimento.

Nel seguente organigramma sono elencati i package e cartelle principali presenti nel progetto. Per ciascuno di essi seguirà una descrizione dei file all'interno. Inoltre, è presente all'interno di ciascun file, una descrizione del codice.



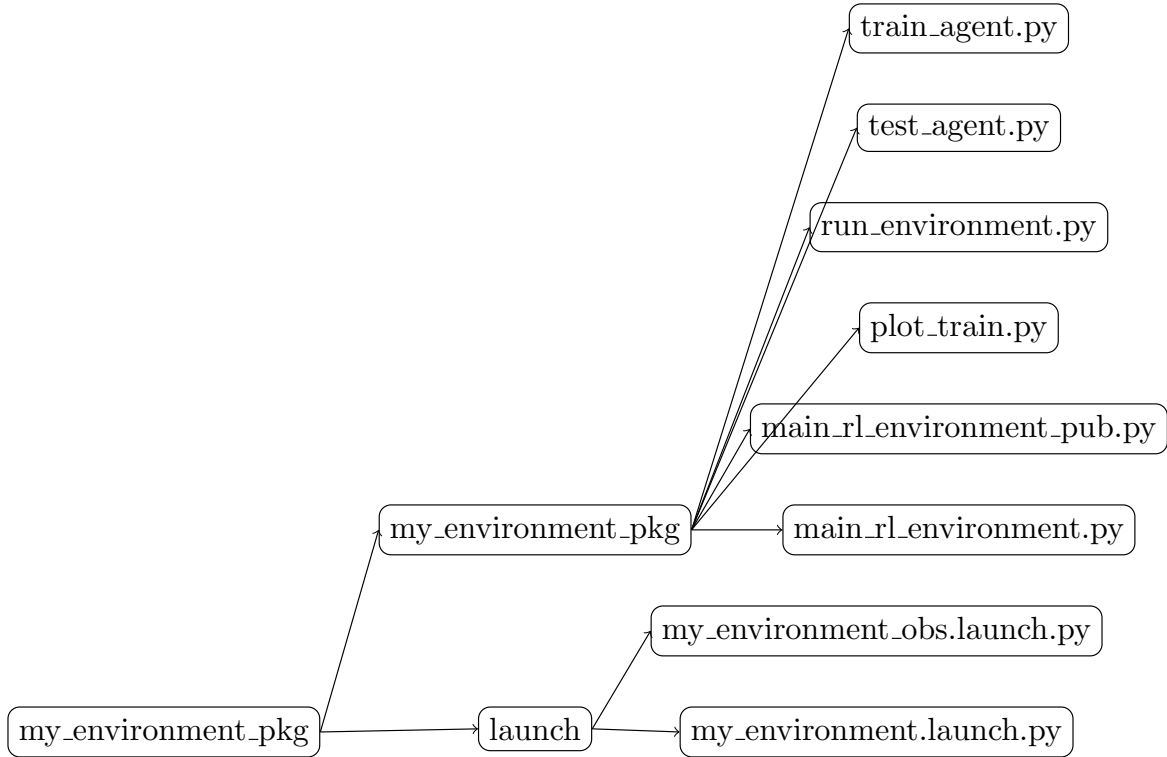
Come si osserva dall'organigramma, il progetto consiste in 3 package "figli" (ovvero il `tm5_900`, `my_sphere_pkg` e `my_obstacle_pkg`) che vengono richiamati e fatti funzionare all'unisono da un package "genitore", ossia `my_environment_pkg`.

Di seguito un listato elenca le funzionalità macroscopiche di ciascun package:

- **tm5_900**: crea il robot all'interno di Gazebo, grazie alle informazioni contenute nei file `.urdf` e `.xacro`.
- **my_sphere_pkg**: crea la sfera target all'interno di Gazebo, grazie alle informazioni contenute nel file `.sdf`.
- **my_obstacle_pkg**: spawna gli ostacoli target all'interno di Gazebo, grazie alle informazioni contenute nel file `.sdf` e `.urdf`.
- **my_environment_pkg**: avvia Gazebo e richiama, all'occorrenza, le strutture definite negli altri package. Si occupa inoltre di creare e avviare il paradigma di allenamento/testing per rinforzo (Reinforcement Learning), salvare i dati di allenamento e log di test.

3.5 Dettaglio dei pacchetti

3.5.1 my_environment_pkg



my_environment.launch.py

Funzione. File di *launch* ROS 2 che avvia un ambiente di simulazione completo. In particolare:

- Avvia **Gazebo** caricando il mondo specifico `my_world.world`.
- Lancia il file di launch del robot **TM5-900**, richiamando il controllore dal file `tm5_900/simple_controller.yaml` (controllo tramite angoli dei giunti) dal file `my_tm5_900_controller.launch.py`.
- Lancia il file di launch della **sfera**, creandola in Gazebo.

In sintesi, orchestra l'avvio simultaneo di simulatore, robot e oggetto target, fungendo da punto di ingresso dell'intero ecosistema.

my_environment_obs.launch.py

Funzione. File di *launch* ROS 2 che orchestra l'avvio di un ambiente di simulazione completo in Gazebo, con l'aggiunta di **ostacoli**. In particolare:

- Avvia **Gazebo** caricando il mondo specifico `my_world.world`.
- Include il file di launch per il robot **TM5-900** e i suoi controller.

- Include il file di launch per creare la **sfera** (`my_sphere_pkg`).
- Include il file di launch per creare gli **ostacoli** (`my_obstacle_pkg`).
- Eventuali nodi aggiuntivi, come bridge ROS-Ignition o listener RL.

In sintesi, avvia simultaneamente simulatore, robot, oggetto target e ostacoli, fornendo un ecosistema completo pronto per train e test.

`main_rl_environment_pub.py`

Funzione. Nodo ROS 2 che funge da interfaccia tra un algoritmo di *Reinforcement Learning* (RL) e un simulatore robotico (Gazebo) con braccio **TM5-900**. Gestisce:

- Monitoraggio dello stato: sottoscrive i topic `JointState`, `ModelStates` e `ContactsState` per conoscere la posizione dei giunti, dell'end-effector e della sfera target.
- Azioni: invia comandi incrementali di movimento ai giunti tramite l'`ActionClient FollowJointTrajectory`, rispettando i limiti fisici del robot.
- Reset: riporta il robot in posizione `Home` e posiziona la sfera target in posizione casuale o manuale.
- Reward: calcola la ricompensa per l'agente RL basata sulla distanza tra end-effector e sfera, includendo penalità per collisioni o uscite dai limiti operativi.
- Interazione dinamica: permette di impostare manualmente la posizione della sfera (`set_target`) e la posizione iniziale del robot (`set_home`) durante il training/test, e di mettere in pausa o riprendere la simulazione (`pause_rl/resume_rl`).

`run_environment.py`

Funzione. Wrapper Gymnasium (OpenAI Gym) per l'ambiente ROS 2/Gazebo gestito da `MyRLEnvironmentNode`. Consente all'agente di Reinforcement Learning di interagire con il manipolatore **TM5-900** attraverso l'interfaccia standard `gym.Env`, disaccoppiando la logica di apprendimento dai dettagli di comunicazione ROS.

Le funzionalità principali includono:

- **Inizializzazione:** viene inizializzato il runtime ROS 2 tramite `rclpy.init()` e creato un nodo applicativo `MyRLEnvironmentNode`. Tale nodo rappresenta l'interfaccia operativa verso Gazebo e verso i controller, occupandosi di inviare comandi ai giunti, leggere lo stato del robot e calcolare le metriche di reward. Sono inoltre definiti lo spazio delle azioni (`Box 6D`, delta di posizione articolare) e lo spazio delle osservazioni (`Box 12D`: posizione end-effector, configurazione dei giunti e posizione del target).
- **Reset:** l'ambiente richiama il metodo `reset_environment_request()` del nodo, che invia una richiesta di ripristino della simulazione (robot in configurazione home e nuovo target). Dopo l'invio della richiesta, il wrapper entra in attesa attiva eseguendo `rclpy.spin_once()` finché il nodo non segnala l'avvenuto aggiornamento dello stato.

tramite il flag `state_updated_flag`. Solo a quel punto viene letta la nuova osservazione iniziale. Se l'osservazione risulta non valida o contiene valori non numerici, viene utilizzato un vettore di zeri come meccanismo di fallback.

- **Step:** l'azione scelta dall'agente viene trasmessa al nodo mediante `action_step_service()`. Anche in questo caso il wrapper attende l'aggiornamento dello stato tramite il medesimo flag, quindi acquisisce la nuova osservazione. Reward e condizione di terminazione sono calcolati dal nodo tramite `calculate_reward_func2()`.
- **Gestione collisioni:** se il nodo segnala una collisione, viene applicata una penalità negativa aggiuntiva (-0.4). In caso di contatto con un ostacolo (`touch`), viene applicata una penalità elevata (-1000) e l'episodio viene terminato.
- **Chiusura:** alla terminazione, il wrapper distrugge il nodo ROS e arresta il runtime tramite `rclpy.shutdown()`.

`train_agent.py`

Funzione. Script principale per l'addestramento di un agente di Reinforcement Learning basato su **Soft Actor-Critic (SAC)** con **Hindsight Experience Replay (HER)**. Si interfaccia con l'ambiente personalizzato `MyGymEnv`, collegato a ROS 2/Gazebo.

Le funzionalità principali includono:

- **Registrazione ambiente:** registra l'ambiente custom in Gymnasium, permettendo l'utilizzo di `gym.make('MyGymEnv')`.
- **Inizializzazione agente:** crea l'agente SAC con dimensioni dello stato e dell'azione dedotte dall'ambiente, e imposta la directory di salvataggio.
- **Ripresa da checkpoint:** se presenti, carica il modello SAC e il replay buffer da checkpoint precedenti.
- **Ciclo principale di training:**
 - Raccolta delle traiettorie episodiche.
 - Applicazione di HER per arricchire il replay buffer.
 - Aggiornamento del modello SAC tramite campionamento batch.
- **Logging:** registra reward medi, successi e tassi di successo in un file CSV ad intervalli di 25 episodi.
- **Salvataggio periodico:** salva modello e replay buffer ogni 50 episodi.

`test_agent.py`

Funzione. Script per il **testing** di un agente SAC addestrato su un braccio robotico in ambiente ROS 2/Gazebo tramite `MyGymEnv`. Esegue N episodi e registra metriche quali reward, distanza dall'obiettivo e configurazione dei giunti.

Funzionalità principali.

- **Caricamento modello:** inizializza l'agente SAC e carica i pesi da un checkpoint specificato.
- **Esecuzione episodi di test:** per ogni episodio, il robot agisce secondo la policy deterministica/stocastica dell'agente.
- **Calcolo metriche:** ad ogni step si calcolano:
 - Distanza iniziale e attuale tra end-effector e target.
 - Ricompensa cumulativa dell'episodio.
 - Stato corrente dei giunti e posizione iniziale (“home position”).
 - Se il target è stato raggiunto.
- **Logging:** scrive i dati raccolti in un file CSV per ogni step, inclusi reward cumulativo, successo globale e tasso di successo.
- **Aggregazione globale:** aggiorna reward totale e conteggio dei successi sull'intero set di episodi di test.

`plot_train.py`

Funzione. Script per l'analisi e la visualizzazione dei dati di monitoraggio dell'addestramento di un braccio robotico in ambiente di Reinforcement Learning. Permette di valutare l'andamento della ricompensa e dei successi episodici nel tempo.

Funzionalità principali.

- **Caricamento dati:** legge il file CSV `Monitoraggio.csv` contenente metriche come reward cumulativo e successi per finestre di 25 episodi.
- **Preparazione dati:** seleziona le colonne di interesse:
 - X: numero episodio
 - Y1: reward medio
 - Y2: numero di successi

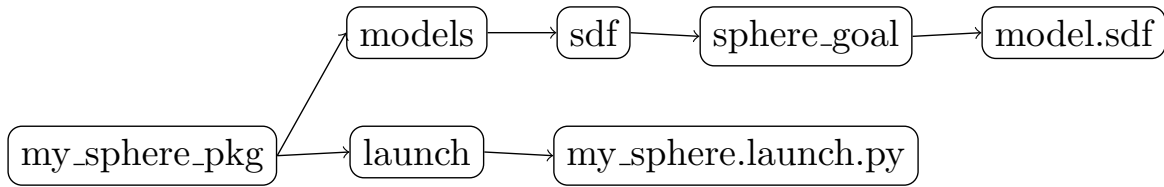
e converte i valori in formato numerico, gestendo eventuali errori.

- **Visualizzazione:** genera due grafici affiancati:
 - Andamento del **reward** rispetto agli episodi.
 - Andamento dei **successi** rispetto agli episodi.

Ogni grafico include etichette degli assi, titolo, griglia e legenda.

- **Personalizzazione:** dimensione della figura, colori e stile dei grafici sono configurabili.

3.5.2 my_sphere_pkg (pacchetto per il goal)



Scopo. Gestisce la creazione della sfera-goal in Gazebo tramite file **SDF** e pubblica la sua **posizione 3D** per l'uso da parte dell'ambiente/controllo. È lanciato con un proprio **launch**, richiamato successivamente dal launch dell'ambiente principale.

Perché separato.

- Disaccoppia la gestione del *goal* dalla logica RL e dal robot.
- Permette di cambiare modello/posa iniziale senza toccare training e controllo.
- Riutilizzabile in altri scenari (*goal* diversi, distribuzioni diverse).

`my_sphere.launch.py`

Funzionamento.

- Recupera il percorso del pacchetto con `get_package_share_directory('my_sphere_pkg')`.
- Definisce il percorso completo del modello SDF: `models/sdf/sphere_goal/model.sdf`.
- Esegue la creazione via `gazebo_ros/spawn_entity.py`, assegnando il nome dell'entità `my_sphere` e posizionandola inizialmente a

$$(x, y, z) = (0, -0.8, 1)$$

- Lancia un nodo ROS 2 che si occupa di caricare la sfera nella simulazione.

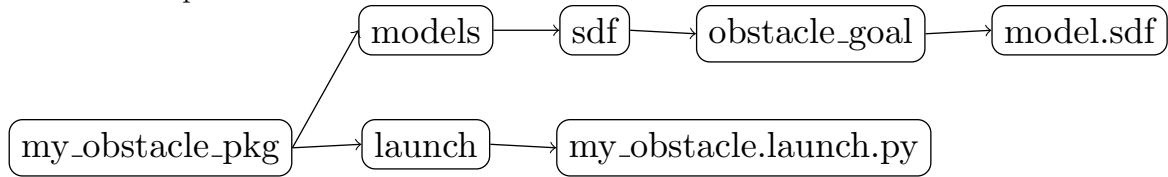
`my_sphere_pkg/models/sdf/sphere_goal/model.sdf`

Caratteristiche principali.

- **Geometria:** sfera con `radius = 0.045` (m).
- **Materiale/colore:** verde (Gazebo/Green) con RGBA impostati.
- **Dinamica:** `static = 0` e `gravity = 0` (il goal resta sospeso all'altezza desiderata); `self_collide = 0`.
- **Inerzia:** massa e inerzia coerenti con oggetto leggero.

3.5.3 my_obstacle_pkg (pacchetto per gli ostacoli)

E' un package equivalente al *my_sphere_pkg*, che crea 3 ostacoli cilindrici attorno all'origine, nell'observation space del braccio robotico.



`obstacle.launch.py`

Funzionamento.

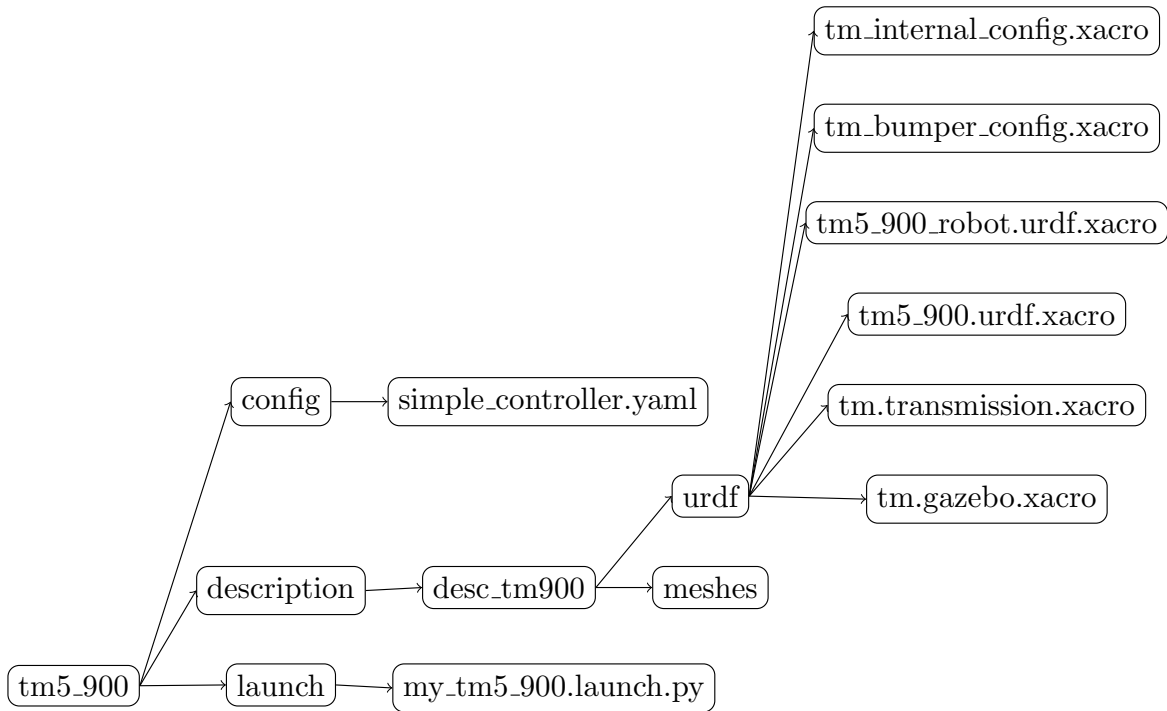
- Recupera il percorso del pacchetto con `get_package_share_directory('my_obstacle_pkg')`.
- Costruisce il percorso assoluto del modello SDF presente in `models/sdf/obstacle_goal/model.sdf`.
- Esegue la creazione dell'entità tramite `gazebo_ros/spawn_entity.py`.
- Il modello viene inserito nella simulazione con nome `my_obstacle`.
- L'output del nodo viene inviato a terminale per facilitare debug e verifica del corretto caricamento.

`my_obstacle_pkg/models/sdf/obstacle_goal/model.sdf`

Caratteristiche principali.

- **Tipologia modello:** insieme di ostacoli cilindrici utilizzati per generare vincoli geometrici durante il moto del manipolatore.
- **Struttura:** il modello è composto da tre link distinti (`cylinder_link_1`, `cylinder_link_2`, `cylinder_link_3`), ciascuno con propria posa nello spazio.
- **Geometria:** ogni ostacolo è un cilindro con `radius = 0.05 m` e `length = 1.3 m`.
- **Posizionamento:**
 - cilindro 1 in `(0.6, 0.35, 0.25)`;
 - cilindro 2 in `(-0.25, 0.55, 0.45)`;
 - cilindro 3 in `(-0.25, -0.433, 0.25)`.
- **Dinamica:** il modello è dichiarato `static=true`, quindi non è soggetto a dinamica o gravità e rappresenta un vincolo fisso nell'ambiente.
- **Collisioni:** ogni link possiede una propria geometria di collisione con parametri di attrito ODE (`mu`, `mu2`) impostati a 1.0. Le collisioni vengono rilevate normalmente e contribuiscono alla logica di penalizzazione dell'ambiente RL.

3.5.4 tm5_900 (descrizione robot & controller)



Scopo. Fornisce la descrizione del manipolatore TM5-900 (URDF/Xacro, mesh, limiti), la configurazione `ros2_control` (controller di traiettoria e joint state) e i file di *launch* per pubblicare lo stato dei giunti e attivare i controller in simulazione.

Perché separato.

- Disaccoppia la *robot definition* dalla logica RL e dai nodi dell'ambiente.
- Rende riutilizzabile il robot in progetti diversi (test, demo, altri task).
- Centralizza limiti, catena cinematica e mapping dei joint.

`simple_controller.yaml`

Funzione. Definisce e configura i controller ROS 2 per il robot in simulazione. Include il `joint_state_broadcaster` per pubblicare lo stato dei giunti e il `joint_trajectory_controller` per eseguire traiettorie sulle sei articolazioni principali del braccio. (I controllori fanno parte della libreria `ros2_control`).

Punti chiave.

- **Aggiornamento:** `update_rate:` 1000 Hz per un ciclo di controllo ad alta frequenza.
- **JointStateBroadcaster:** pubblica continuamente le posizioni e velocità dei giunti tramite `joint_state_broadcaster/JointStateBroadcaster`.

- **Trajectory Controller:** configurato con guadagni PID specifici per ogni giunto, interfacce di comando in posizione (`command_interfaces: position`) e interfacce di stato (`state_interfaces: position, velocity`).
- **Constraint e Rate:** `state_publish_rate` e `action_monitor_rate` settati a 1000 Hz, con vincoli per velocità massime e tempo di arrivo al target.
- **Gazebo Integration:** usa `gazebo_ros2_control/GazeboSystem` come robot hardware per simulazione.

Motivazioni.

- Fornire un controllo stabile e rapido dei giunti in simulazione garantendo `/joint_states` aggiornati e la disponibilità dell'interfaccia `FollowJointTrajectory`.
- Separare la definizione dei controller dalla pipeline di lancio semplifica la manutenzione e la sperimentazione con differenti parametri PID o frequenze di update.

Accortezze.

- Assicurarsi che il `joint_state_broadcaster` sia avviato prima del `joint_trajectory_controller`, oppure utilizzare gli *spawner* con *event handlers* per sequenzializzare correttamente il caricamento.
- Parametrizzare il file YAML via `LaunchConfiguration` consente di modificare facilmente i guadagni, le interfacce o le frequenze senza toccare il codice del launch.

`tm5_900_robot.urdf.xacro`

Funzione. Punto d'ingresso della descrizione robotica: include i moduli comuni (`tm5_900.urdf.xacro`, parametri Gazebo/trasmissioni) e istanzia il macro-robot `tm5_900_robot`. Aggiunge inoltre il frame `world` e la giunzione fissa verso `base`, introducendo una rotazione iniziale ($rp_y = [0.5\pi, 0, 0]$) per far sì che il robot sia in posizione verticale.

Note tecniche.

- La rotazione $\frac{\pi}{2}$ sull'asse x allinea la base al frame del mondo di Gazebo;
- La lista delle *state interfaces* del giunto `wrist_3_joint` (end-effector) è in ordine `velocity`, `position`, `effort` (tutti gli altri joint usano `position`, `velocity`, `effort`): funziona comunque, ma si può uniformare per pulizia.
- Il blocco `<ros2_control>` all'interno dell'URDF/Xacro definisce l'astrazione dell'hardware del robot, specificando per ciascun giunto le interfacce di comando e di stato disponibili. Nel caso in esame viene utilizzata la *command interface* di tipo `position`, mentre come *state interfaces* sono esposte `position`, `velocity` ed `effort`, rendendo tali informazioni accessibili ai controller e pubblicabili, ad esempio, sul topic `/joint_states`. Il collegamento tra l'infrastruttura `ros2_control` e la simulazione fisica in Gazebo è

realizzato tramite il plugin `gazebo_ros2_control`, caricato nel tag `<gazebo>`. Questo plugin istanzia un hardware simulato, traduce i comandi provenienti dai controller ROS 2 in azioni sui giunti del modello e restituisce gli stati aggiornati alla parte ROS. La configurazione operativa dei controller (giunti coinvolti, parametri PID, frequenze di aggiornamento, ecc.) è affidata al file indicato nel parametro `parameters`, in questo caso `config/simple_controller.yaml`.

`tm5_900.urdf.xacro`

Funzione. Definisce il macro `tm5_900_robot` che compone:

- **catena cinematica** completa (link & joint), con limiti giunto, attriti e smorzamenti;
- **inerzie** tramite macro (`cylinder_inertial_y/_z`) con masse plausibili;
- **trasmissioni** (`xacro:tm_arm_transmission`);
- **config Gazebo** (`xacro:tm_arm_gazebo`, `xacro:tm_internal_config`, `xacro:tm_bumper_ctr_sensor`);
- **mesh visual/collision** per ciascun link.

Scelte di modellazione.

- Limiti giunto: ampi ma realistici; `safety_controller` attivabile via parametro per margini “soft”.
- **dynamics** sui giunti (damping/friction) per attenuare oscillazioni in simulazione.
- Frame `tool0` coincidente con la flangia (`flange_link`) per un TCP “all zero” comodo nel calcolo dell’EE pose.

Nota pratica. Se noti *self-hit* troppo frequenti (urti tra link vicini), valuta:

- di alleggerire le collisioni (*collision meshes* più sottili) o
- introdurre un’**allowed collision matrix** (in MoveIt) o filtri di contatto in Gazebo (bitmask) per coppie “fisiologicamente vicine”.

Implicazioni.

- Aumenta la fedeltà della simulazione rispetto alle auto-collisioni, ma può indurre **reset frequenti** se la postura nominale porta link molto vicini.
- Se il task non richiede contatti interni, valutare di impostare **false** su alcuni link o usare *collision filtering* (bitmask) per escludere coppie “sicure”.

`tm.gazebo.xacro`

Funzione. Il macro `tm_arm_gazebo` introduce le configurazioni specifiche per la simulazione in Gazebo associate ai link del manipolatore. Per ciascun elemento strutturale principale del braccio viene abilitata l’opzione di *self-collision*, consentendo al motore fisico di rilevare eventuali contatti tra parti appartenenti allo stesso robot.

Note tecniche.

- L'attivazione di `selfCollide` aumenta il realismo della simulazione, evitando compenetrazioni non fisiche tra i link;
- Il controllo delle auto-collisioni è particolarmente rilevante durante pianificazioni di movimento in spazi ristretti;
- L'abilitazione può comportare un lieve aumento del carico computazionale, dovuto al maggior numero di verifiche richieste dal motore fisico.

`tm.transmission.xacro`

Funzione. Macro `tm_arm_transmission`: dichiara, per ogni giunto, la **transmission** di tipo `hardware_interface/PositionJointInterface` e l'attuatore corrispondente.

Perché Position. Consente di comandare il trajectory controller in **posizione** (interfaccia più semplice/stabile in simulazione). In futuro si possono introdurre interfacce `velocity/effort` per compiti dinamici.

`tm_internal_config.xacro`

Funzione. Imposta parametri “fisici” per Gazebo (per link base e segmenti): `kp`, `kd` del contatto, attriti `mu1/mu2`, `maxVel` e `minDepth`. Servono a stabilizzare i contatti ed evitare compenetrazioni/oscillazioni.

Osservazioni.

- Valori molto alti ($kp = 100,000$) irrigidiscono il contatto ma possono introdurre jitter; se si nota instabilità, valutare di ridurli o di aumentare `maxStepSize`/ridurre `realTimeUpdateRate` nel mondo.
- Uniformare questi parametri su tutti i link evita *discontinuità* di risposta al contatto.

`tm_bumper_config.xacro`

Funzione. Aggiunge **sensori di contatto** (bumper) su `base_link`, `flange_link` e sugli altri link principali, usando il plugin `libgazebo_ros_bumper.so`. I topic sono **namespaced** sotto `/contact_sensor` con remapping `bumper_states:=...` per link.

Uso nell'ambiente RL.

- Segnala *self-hit* o contatti con il suolo; l'ambiente può usarli per penalità/terminazioni e reset.
- Gli `update_rate` molto elevati (100–1000 Hz) sono reattivi ma più pesanti: più il carico aumenta, più sarebbe corretto abbassarli.

`my_tm5_900_controller_launch.py`

Funzione. Lancia l'intera pipeline del robot TM5-900 in simulazione Gazebo: genera l'URDF a partire dal file `.xacro`, avvia il nodo `robot_state_publisher`, crea il modello in Gazebo usando la descrizione pubblicata su `robot_description` e carica/avvia i controller ROS 2 (`joint_state_broadcaster` e `joint_trajectory_controller`).

Punti chiave.

- **Xacro** → **URDF**: utilizza `ParameterValue(Command(['xacro ', xacro_file]))` per generare l'URDF on-the-fly e popolare il parametro `robot_description`.
- **Creazione in Gazebo**: `gazebo_ros/spawn_entity.py` con `-topic robot_description` inserisce il modello nella simulazione.
- **Controller**: caricati ed avviati tramite comandi `ros2 control load_controller --set-state start`, garantendo la disponibilità dei topic `/joint_states` e dell'interfaccia `FollowJointTrajectory`.

Motivazioni.

- Separare la generazione della descrizione URDF dal lancio del robot mantiene la pipeline modulare e riproducibile.
- L'avvio esplicito dei controller assicura che la simulazione possa leggere lo stato dei giunti e inviare comandi di traiettoria in modo stabile.

Accortezze.

- In alcuni setup conviene serializzare il caricamento dei controller: prima `joint_state_broadcaster`, poi `joint_trajectory_controller`, oppure usare gli *spawner* con *event handlers*.
- Parametrizzare il percorso del file `.xacro` e i nomi dei controller tramite `LaunchConfiguration` aumenta la flessibilità del launch.

3.6 Funzionamento complessivo

All'avvio, il *launch file* principale inizializza Gazebo con il modello del TM5-900. Successivamente vengono avviati i nodi di gestione ambiente e target. Infine, il nodo agente inizia a interagire con il sistema. L'integrazione tra i componenti è garantita dalla sincronizzazione dei messaggi ROS: l'agente attende di ricevere uno stato valido prima di inviare la prima azione, garantendo che il robot non esegua movimenti bruschi all'avvio.

4 Test e risultati

In questo capitolo vengono presentati i risultati sperimentali ottenuti durante l'addestramento del braccio robotico manipolatore. L'obiettivo delle sperimentazioni è stato valutare l'efficacia di diverse strategie di apprendimento per il raggiungimento di un target nello spazio 3D, analizzando la convergenza dell'algoritmo e la stabilità della policy appresa.

In particolare, sono state condotte due campagne di test principali per confrontare approcci diversi alla complessità del task:

1. **Curriculum Learning:** un approccio incrementale basato sulla difficoltà crescente del task, utilizzando l'algoritmo *Soft Actor-Critic* (SAC) puro.
2. **Success Threshold Annealing:** un approccio basato sulla riduzione progressiva della soglia di successo, combinando SAC con *Hindsight Experience Replay* (HER).

4.1 Configurazione Sperimentale

Tutti gli esperimenti descritti in seguito sono stati condotti in un ambiente di simulazione **privo di gravità**. Questa scelta è derivata dall'analisi delle problematiche riscontrate in fase preliminare (si veda la Sezione 4.4), dove l'attivazione della gravità introduceva instabilità nei controllori PID dei giunti tali da compromettere la coerenza del segnale di reward e l'apprendimento stesso.

4.1.1 Funzioni di Reward

Per le due campagne di test sono state adottate funzioni di ricompensa distinte, coerenti con la natura degli algoritmi utilizzati.

Reward A (Test 1: Curriculum Learning)

Nel primo test, non utilizzando HER, è stato necessario progettare una funzione di reward *densa* (reward shaping) per guidare l'agente verso l'obiettivo. Il reward totale R_t è calcolato come somma di diverse componenti:

- **Distanza:** Una penalità proporzionale alla distanza cartesiana d dall'obiettivo (es. $-0.4 \cdot d$), scalata in base alla vicinanza.
- **Successo:** Un bonus significativo (+100) in caso di raggiungimento del target, scalato in base alla velocità di esecuzione $(1 - \frac{steps}{max_steps})$.
- **Shaping Dinamico:** Un premio (+0.3) se la distanza corrente è minore della migliore distanza registrata nell'episodio (progresso), altrimenti una piccola penalità per scoraggiare lo stallo.
- **Penalità:** Valori negativi per collisioni, self-collision, velocità eccessiva dei giunti o collisione dell'end-effector con il suolo.

Reward B (Test 2: Threshold Annealing + HER)

Nel secondo test, l'uso di HER ha permesso di semplificare drasticamente il segnale di feedback, adottando una struttura *sparsa*:

- **Penalty per step** (-10): Assegnata a ogni passo temporale in cui l'obiettivo non viene raggiunto.
- **Penalty per collisione** (-1100): Una forte penalità in caso di collisione.
- **Reward di successo** ($+10$): Assegnato esclusivamente quando la distanza Euclidea scende sotto la soglia (*threshold*) corrente.

4.2 Test 1: Curriculum Learning con SAC

La prima strategia analizzata ha implementato un sistema di spawn del target a difficoltà crescente. L'area in cui la sfera obiettivo poteva apparire era definita da una serie di volumi (*boxes*) preconfigurati. Il passaggio al livello successivo era governato da una logica automatica che monitorava il tasso di successo e la distanza media sui 50 tentativi precedenti.

Le specifiche delle box utilizzate sono state le seguenti:

- **Livello Facile:** $x, y \in [-0.60, 0.60]$, $z \in [0.75, 0.90]$.
- **Livello Medio:** $x, y \in [-0.40, 0.40]$, $z \in [0.75, 0.95]$.
- **Livello Full Range:** $x, y \in [-0.50, 0.50]$, $z \in [0.60, 0.95]$.

4.2.1 Analisi Critica dei Risultati

L'andamento del training è illustrato nelle Figure 4.1, 4.2 e 4.3, che mostrano rispettivamente la distanza minima raggiunta, il tasso di successo e il reward cumulativo per episodio.

Analizzando congiuntamente i tre grafici, emerge chiaramente la dinamica dell'apprendimento legata al curriculum:

1. **Apprendimento iniziale (Livello Facile):** Nella fase iniziale (fino all'episodio 800 circa), si osserva un trend nettamente positivo. La distanza minima decresce, il tasso di successo sale rapidamente verso il 100% e il reward aumenta. Questo indica che il robot apprende velocemente a raggiungere i target nel volume ristretto e semplificato.
2. **Primo cambio di livello (Livello Medio):** Intorno all'episodio 900, si verifica un crollo delle prestazioni (la distanza aumenta, i successi e il reward crollano). Questo segna l'attivazione del livello "Medio". Dopo una fase di riadattamento, l'agente riesce a recuperare, ottenendo nuovamente valori buoni che permettono di soddisfare le condizioni per il passaggio successivo.
3. **Stallo e Overfitting (Full Range):** Il secondo scatto di livello provoca un nuovo decadimento dei risultati. Tuttavia, a differenza della fase precedente, qui non si osserva un miglioramento costante. Le prestazioni rimangono oscillanti e mediocri.

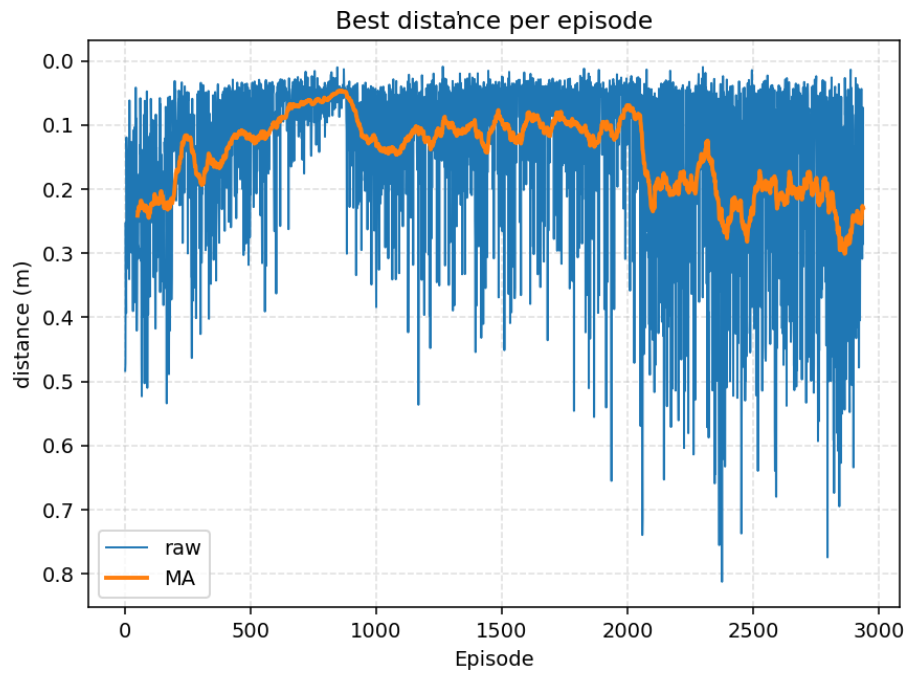


Figure 4.1: Andamento della minima distanza raggiunta per episodio (linea blu) e relativa media mobile (linea arancione). Si notano i cali di prestazione (aumento della distanza) in corrispondenza dei cambi di livello.

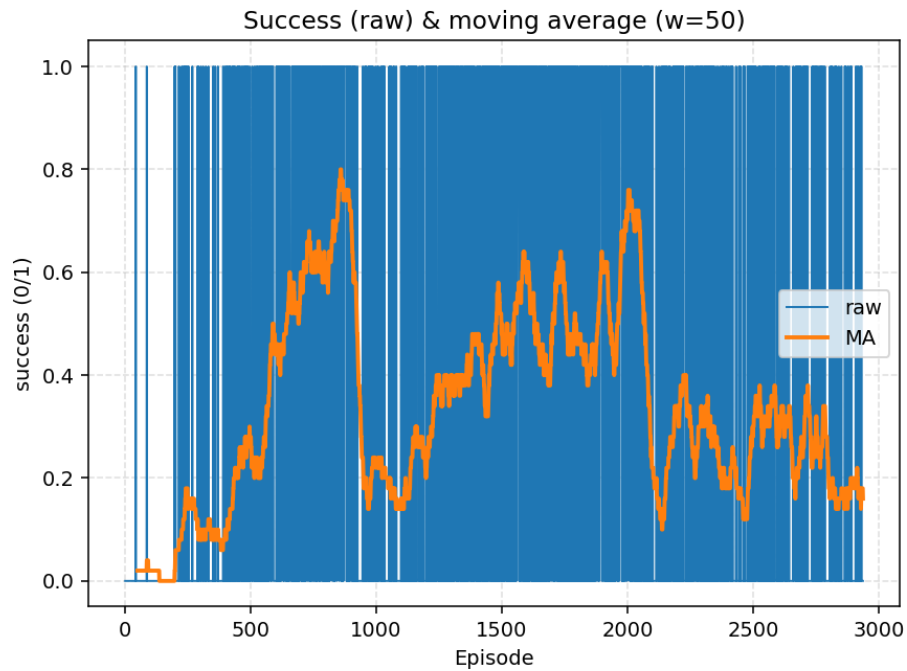


Figure 4.2: Numero di successi per episodio (1 = successo, 0 = fallimento). La linea arancione rappresenta la media mobile del tasso di successo.

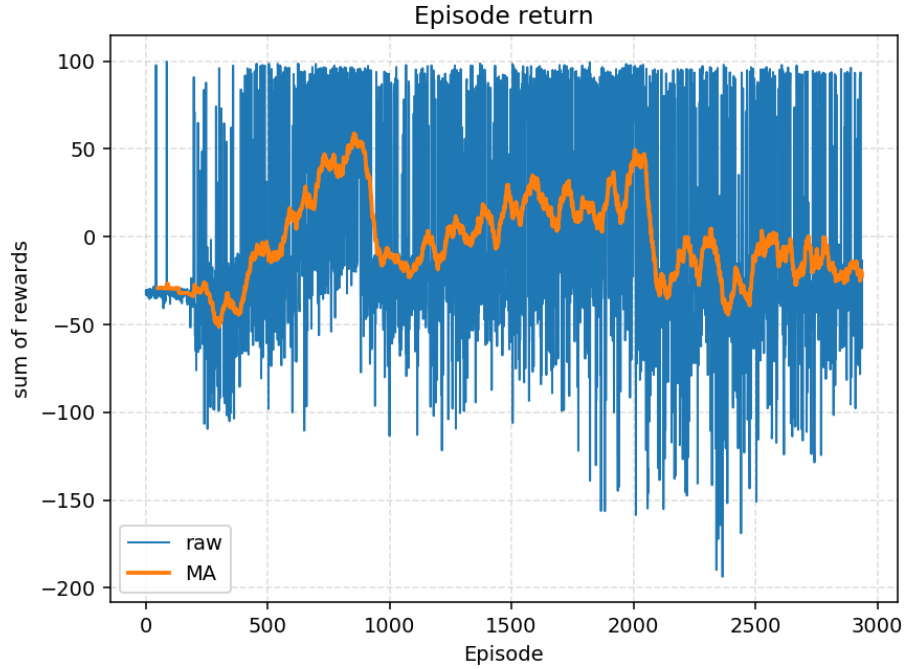


Figure 4.3: Andamento del reward cumulativo (return) per episodio. La linea arancione evidenzia il trend medio.

La causa di questo comportamento è riconducibile alla sovrapposizione dei volumi di spawn descritta in precedenza. Poiché i livelli più difficili includono anche le zone del livello facile, l'agente continua a ottenere successi sporadici resolvendo i compiti che già conosce (i punti vicini), come evidenziato dal grafico del reward che si stabilizza in una fascia intermedia. L'agente sembra aver trovato una strategia sub-ottimale o "zona sicura": evita le forti penalità e raccoglie reward positivi sui target facili, ma non è stimolato a sufficienza per esplorare e risolvere i target nelle nuove zone più complesse, risultando in una mancata generalizzazione della policy.

4.3 Test 2: Success Threshold Annealing con SAC + HER

La seconda campagna di test ha utilizzato l'algoritmo SAC potenziato da HER (*Hindsight Experience Replay*), adottando una strategia di *Threshold Annealing*: la difficoltà non è data dalla posizione del target (che viene generato in tutto lo spazio di lavoro), ma dalla precisione richiesta per considerare il task "risolto".

4.3.1 Analisi dei Risultati

La Figura 4.4 mostra l'andamento del reward e del tasso di successo durante il training.

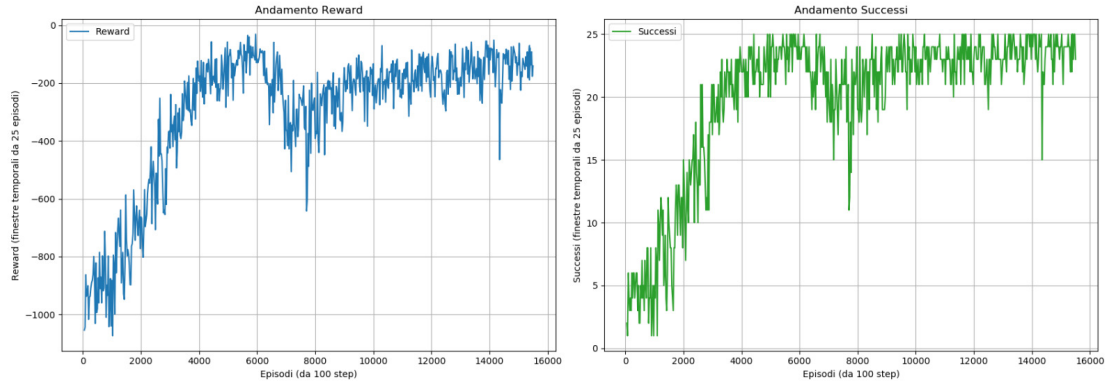


Figure 4.4: *Andamento del reward e dei successi durante il training con Threshold Annealing.*

L'andamento dei successi presenta un chiaro trend crescente. Si possono osservare due fenomeni principali:

1. **Picchi negativi e Adattamento:** I cali improvvisi nelle prestazioni, visibili ad esempio intorno all'episodio 7000 e successivamente al 14000, corrispondono ai momenti in cui la *threshold* viene ridotta (es. da 0.15m a 0.12m, e poi a 0.06m). Quando la soglia si restringe, il modello, che prima otteneva il successo, improvvisamente non soddisfa più il criterio. Questo "shock" costringe l'agente a riadattare la policy per migliorare la precisione. Dopo una fase transitoria, i successi tornano a stabilizzarsi su valori alti.
2. **Prestazioni a Regime:** Già intorno ai 4000 episodi (con threshold ancora permissiva di 0.15m), il modello raggiunge buone prestazioni, con oltre 20 successi su 25 episodi. Dopo i 14000 episodi, con la threshold minima di 0.06m, il modello dimostra robustezza raggiungendo una media stabile di circa 24 successi su 25.

Il contributo dell'algoritmo **HER** si è rivelato fondamentale: permettendo al modello di apprendere anche dagli episodi fallimentari (reinterpretandoli come successi rispetto a target virtuali), ha stabilizzato il gradiente e accelerato la convergenza in un ambiente a ricompensa sparsa.

4.3.2 Validazione Finale

Al termine della fase di addestramento, per confermare la robustezza della policy appresa, è stato eseguito un test di validazione. In questa fase, l'agente ha operato su un set fisso di target distribuiti nello spazio di lavoro (non generati casualmente) e in modalità *inferenza*, ovvero con i pesi della rete congelati e senza l'aggiunta di rumore esplorativo alle azioni.

I risultati confermano che l'agente è in grado di pianificare efficacemente una traiettoria verso i punti 3D assegnati, mantenendo l'errore finale (distanza Euclidea dall'obiettivo) stabilmente al di sotto della soglia di 0.06 metri, come richiesto dai criteri di successo dell'ultimo step del curriculum.

4.4 Problematiche Riscontrate e Lezioni Apprese

4.4.1 Problematiche

- **Gravità e Dinamica:** Come anticipato, l'addestramento con gravità attiva in Gazebo non ha prodotto risultati soddisfacenti. I controllori dei giunti faticavano a compensare la gravità mantenendo le posizioni target, introducendo un "rumore" fisico che rendeva incoerente il segnale di reward (l'agente sceglieva l'azione corretta, ma il braccio non la eseguiva fedelmente). Disabilitare la gravità ha permesso di isolare e validare la logica di RL.
- **Ostacoli:** La gestione degli ostacoli non è stata ancora validata completamente e necessita di ulteriori iterazioni per verificarne l'impatto sulla convergenza.

4.4.2 Lezioni Apprese

Dalle sperimentazioni sono emersi i seguenti punti chiave per il design di futuri sistemi di controllo robotico basati su RL:

- **Progettazione dei Volumi nel Curriculum:** L'esperimento ha evidenziato come la semplice espansione del volume di spawn non sia sufficiente se i nuovi livelli continuano a includere le zone "facili" precedentemente apprese. Questo porta l'agente a massimizzare il reward risolvendo ripetutamente i task noti, ignorando le nuove aree più complesse. Una progettazione robusta richiede che i livelli avanzati escludano esplicitamente le regioni spaziali già padroneggiate, forzando l'agente a generalizzare la policy su tutto lo spazio di lavoro.
- **Efficacia dell'Annealing + HER:** La combinazione di una soglia di successo decrescente ($0.15\text{m} \rightarrow 0.06\text{m}$) con HER è risultata la strategia vincente, garantendo un apprendimento continuo e denso di reward.
- **Monitoraggio:** L'importanza di una valutazione robusta su un set fisso di target di validazione e il logging continuo dei dati (es. *Monitoraggio.csv*) per distinguere il vero apprendimento dal rumore statistico.

L'agente ha dimostrato di saper generalizzare il compito, riuscendo a raggiungere il target anche quando questo viene riposizionato in punti diversi dello spazio di lavoro (Domain Randomization sulla posizione di home). La stabilità del training è migliorata nettamente rispetto al Test 1, confermando l'efficacia dell'approccio SAC+HER in assenza di disturbi gravitazionali non compensati.

5 Conclusioni e sviluppi futuri

Il presente lavoro ha permesso di sviluppare e validare una pipeline completa di *Deep Reinforcement Learning* per il controllo del manipolatore collaborativo TM5-900 in ambiente simulato ROS 2.

L'obiettivo principale era quello di addestrare un agente intelligente in grado di pianificare traiettorie nello spazio dei giunti per raggiungere un target cartesiano, superando le difficoltà intrinseche di un task a ricompensa sparsa.

L'integrazione dell'algoritmo *Soft Actor-Critic* (SAC) con la tecnica *Hindsight Experience Replay* (HER) si è dimostrata l'architettura vincente. Questa combinazione ha permesso di massimizzare l'efficienza nell'uso dei dati, trasformando i numerosi tentativi fallimentari iniziali in preziose opportunità di apprendimento.

Risultati Conseguiti

Dalle campagne di test effettuate emergono le seguenti conclusioni principali:

- **Gestione della Complessità:** Il confronto tra le due strategie di curriculum ha evidenziato i limiti di un approccio basato sulla sola espansione volumetrica del task (*Curriculum Learning* spaziale), che tende a soffrire di overfitting sulle regioni già esplorate. Al contrario, la strategia di *Success Threshold Annealing* si è rivelata determinante per stabilizzare l'apprendimento, guidando l'agente da una policy grossolana a una di precisione senza bias spaziali.
- **Dinamica e Controllo:** L'utilizzo di controllori di posizione standard in presenza di gravità ha mostrato limiti significativi nel tracciamento delle azioni ad alta frequenza generate dalla rete neurale. La disabilitazione della gravità in simulazione è stata una scelta necessaria per disaccoppiare la logica di apprendimento dai disturbi introdotti dal loop di controllo di basso livello.
- **Generalizzazione:** L'uso della *Domain Randomization*, implementata attraverso la variazione casuale del punto di creazione del target nell'intero workspace, ha garantito che l'agente non memorizzasse singole traiettorie, ma apprendesse una policy robusta e generalizzabile.

Sviluppi Futuri

Per colmare il divario tra simulazione e realtà e migliorare le prestazioni complessive, si identificano le seguenti linee di ricerca future:

1. **Controllo Avanzato a Basso Livello:** Per abilitare il training con gravità attiva (essenziale per il deployment reale), è necessario intervenire sul controllore del robot.

Si suggerisce l'implementazione di un controllo di coppia (*Torque Control*) o l'ottimizzazione dei guadagni PID con termini di *feedforward* per la compensazione dinamica della gravità.

2. **Gestione degli Ostacoli:** Sebbene l'infrastruttura software per la gestione degli ostacoli sia stata predisposta, la sua validazione richiede ulteriori iterazioni. Si propone di introdurre gli ostacoli nell'ambiente in modo incrementale, penalizzando le collisioni in modo graduale per evitare che l'agente converga su minimi locali eccessivamente conservativi (es. immobilità per paura di collisione).
3. **Ottimizzazione degli Iperparametri:** Un fine-tuning dei parametri di SAC, in particolare il coefficiente di entropia α (magari rendendolo apprendibile dinamicamente con il progressivo raggiungimento del target), potrebbe velocizzare la convergenza nelle prime fasi.
4. **Analisi delle Prestazioni e Metriche Derivate:** Sfruttando i dati già presenti nei log, si raccomanda lo sviluppo di strumenti di analisi ex-post più raffinati. In particolare:
 - Calcolo del *Delta distanza* dell'EE dal target ($\Delta d_t = d_t - d_{t-1}$, calcolato per ogni step di simulazione $t > 0$, con t equivalente a un singolo step del robot) e dell'*improvement rate* per misurare puntualmente il progresso locale.
 - Analisi della variazione angolare delle traiettorie per quantificare la fluidità del movimento step-by-step su un episodio.
 - Generazione di *Heatmap tridimensionali* del workspace per visualizzare le regioni a bassa performance e implementazione di un *Hard-case buffer* per riproporre all'agente gli scenari più critici.
5. **Estensione dell'Architettura di Logging:** Per potenziare l'osservabilità del sistema durante il training, si suggerisce di arricchire il logger con:
 - **Tracciamento Eventi:** Registrazione in linea di comandi operativi da CLI (pause, resume, set_target) e codifica esplicita della causa di terminazione (collisione, timeout) per diagnosi automatiche.
 - **Metriche Temporali e Interne:** Inserimento di timestamp reali (per stimare le latenze di calcolo oltre ai semplici step) e salvataggio dei valori interni della rete SAC (es. Q-values, loss) per correlare l'andamento della ricompensa con la convergenza della rete.
6. **Deployment su Hardware Reale:** Una volta risolte le problematiche di controllo dinamico, il passo successivo naturale sarà il trasferimento della policy sul robot fisico TM5-900, validando la sicurezza e la precisione in uno scenario reale.

Bibliografia

- [1] Gazebo classic tutorials. <https://classic.gazebosim.org/tutorials>. Consultato durante lo sviluppo del progetto.
- [2] Ros 2 foxy fitzroy tutorials. <https://docs.ros.org/en/foxy/Tutorials.html>. Documentazione ufficiale ROS 2.
- [3] Marcin Andrychowicz et al. Hindsight experience replay. *arXiv preprint arXiv:1707.01495*, 2017.
- [4] Tuomas Haarnoja et al. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.
- [5] Nicola Misurati. Tm5-900 ros 2 implementation. https://github.com/nicolamisurati/tm5-900_ros/tree/main. Repository GitHub contenente i pacchetti per il robot TM5-900.
- [6] David Valencia. Robotic arm environment for reinforcement learning. https://github.com/dvalenciar/robotic_arm_environment?tab=readme-ov-file. Repository GitHub di riferimento per l'ambiente di simulazione.